Museum Guard Shift – Final Report

Group Members: Dmitri Lvovich, Raphaël Michnik

Problem Summary

A guard starts at room S in a museum whose rooms form a weighted tree. Exactly K rooms contain alarms. Each alarm i is in room r_i; handling it takes a_i time after arrival, and must be completed by deadline d_i. The guard must handle *all* alarms and minimize the total time to do so, or report `IMPOSSIBLE`.

Intended Solution

The intended solution has two phases.

Phase 1: Distance Preprocessing

Because the graph is a weighted tree, the shortest-path distance between any two nodes u and v equals

$$\text{dist}(u, v) = \text{distRoot}[u] + \text{distRoot}[v] - 2 \cdot \text{distRoot}[\text{LCA}(u, v)],$$

where distRoot$[\cdot]$ stores weighted distances from the chosen root, and LCA is the lowest common ancestor. We root the tree at S, compute root-distances in $O(N)$ via DFS, and build an LCA structure in $O(N \log N)$ (or $O(N)$ with the Farach-Colton–Bender RMQ trick). This lets us answer each pairwise distance query among the $K + 1$ important nodes in $O(\log N)$ or $O(1)$, giving an $O(K^2 \log N)$ or $O(K^2)$ precomputation.

Phase 2: Bitmask Dynamic Programming

Let

$$dp[\text{mask}][i]$$

be the earliest time the guard can finish handling all alarms in `mask` while ending at alarm i.

Initialization.

$$dp[1 \ll i][i] = \text{dist}(S,\, r_i) + a_i \quad \text{if } \text{dist}(S,\, r_i) + a_i \le d_i.$$

Transition. For each state (mask, i) with $dp[\text{mask}][i] < \infty$ and each unvisited alarm $j \notin$ mask:

$$\text{new_time} = dp[\text{mask}][i] + \text{dist}(r_i,\, r_j) + a_j.$$

If new_time $\le d_j$, update $dp[\text{mask} \cup \{j\}][j] = \min(\cdots, \text{new_time})$.

Answer. $\min_i \; dp[(1 \ll K) - 1][i]$, or `IMPOSSIBLE` if all are ∞.

Correctness. Optimal substructure holds: any optimal full schedule consists of an optimal partial schedule for some subset, plus one valid transition. The DP keeps only the earliest finish time for each (mask, i) pair, dominating every slower path to the same state.

Complexity. $O(2^K K^2)$ for the DP, plus $O(N \log N + K^2)$ for preprocessing. Feasible for $N \leq 200\,000$ and $K \leq 18$ (the DP table has $2^{18} \times 18 \approx 4.7 \times 10^6$ entries, each with up to 18 transitions).

Incorrect and Inefficient Solutions

Wrong Answer: Greedy by Earliest Deadline

File: `submissions/wrong_answer/cppwrong.cpp`

This solution sorts alarms by deadline (smallest first) and visits them greedily in that order, using LCA-based distances. It fails because it ignores the guard's current position: choosing the earliest-deadline alarm can send the guard far away, making a second tight-deadline alarm unreachable even when a valid ordering exists.

Failing case: `secret07_deadline_greedy_trap.in` — the greedy outputs `IMPOSSIBLE` while the correct answer is 6. The tree is a 3-node chain 1–2–3; alarm 2 (in room 2) has deadline 7 and alarm 3 (in room 3) has deadline 6. The greedy visits room 3 first (lower deadline), traveling 4 then spending 1, finishing at 5. It then travels back to room 2 (distance 2), finishing at 8> 7. The optimal order visits room 2 first: finish at $2 + 1 = 3 \leq 7$, then room 3: finish at $3 + 2 + 1 = 6 \leq 6$.

Time Limit Exceeded: Brute-Force Permutation Search

File: `submissions/time_limit_exceeded/cpptle.cpp`

This solution correctly precomputes all pairwise distances with BFS, but instead of DP it tests every one of the $K!$ permutations of alarm visits. For $K = 18$, this is $18! \approx 6.4 \times 10^{15}$ iterations — completely infeasible within any reasonable time limit. The solution is correct on small inputs (it agrees with the accepted solutions on all test cases with small K), but will TLE on any test with $K \geq 12$ or so.

Probable Errors Contestants Could Make

1. **Greedy by nearest room.** Choosing the closest unhandled alarm can fail: a slightly farther alarm may have a much tighter deadline.
2. **Greedy by earliest deadline.** Travel time is not uniform, so the classic EDF scheduling rule does not apply here.
3. **Recomputing distances naively.** Running BFS/DFS for every alarm pair from scratch is $O(KN)$ per pair, and $O(K^2 N)$ total. For $N = 200\,000$ and $K = 18$ that is $\approx 6.5 \times 10^{10}$ operations — TLE.
4. **Treating it as ordinary shortest path.** The guard must visit all K alarms in *some* order respecting deadlines; this is not a single-source shortest-path instance.

5. **Checking the deadline at arrival, not at completion.** The problem states that the alarm is handled only after spending a_i time at the panel. A common off-by-one is to check arrival_time $\leq d_i$ rather than arrival_time $+ a_i \leq d_i$.

6. **Integer overflow.** Edge weights up to 10^9, paths up to $N - 1$ edges long, plus handling times up to 10^9: distances can exceed 2×10^{17}, requiring 64-bit integers throughout.

Design Choices

1. **Tree structure with small K.** The tree provides a clean distance model while keeping input sizes large. Small K makes the bitmask DP the natural intended solution, without any polynomial alternative.

2. **Weighted edges.** Uniform weights would allow BFS-like greedy reasoning; weights force competitors to use actual travel times.

3. **Start room may be alarmed.** Handled naturally by the DP ($\text{dist}(S, r_i) = 0$ when $r_i = S$); tested in `secret04_start_room_alarmed`.

4. **Guard need not return to start.** Adding a return leg would add complexity without algorithmic depth; omitted for clarity.

5. **Deadlines can be large ($\leq 10^{15}$).** Forces 64-bit arithmetic and tests that implementations do not overflow.

Test Cases

Sample Cases (2 files)

`sample01` A 6-node tree, $K = 3$ alarms, start at room 1. All deadlines are comfortably met by one specific visiting order (rooms $4 \to 6 \to 5$); answer is 26. Tests basic correctness and the sample explanation.

`sample02` Same tree and alarms, but room 4's deadline is reduced from 6 to 4. Since the earliest possible finish time for room 4 is $4+1 = 5 > 4$, the answer is `IMPOSSIBLE`. Tests the impossible case.

Secret Cases (15 files)

`secret01_no_alarms` $K = 0$; output must be 0. Tests the degenerate base case that some solutions may not handle.

`secret02_single_exact` $K = 1$; deadline equals the exact arrival-plus-handling time. Tests the $\leq$ boundary condition (equal is feasible).

`secret03_single_impossible` $K = 1$; deadline is one unit below the minimum finish time. Tests the $\leq$ boundary condition (over by one is impossible).

`secret04_start_room_alarmed` The starting room is one of the alarmed rooms (travel distance $= 0$). Tests implementations that compute $\text{dist}(S, r_i)$ for the starting-room alarm.

`secret05_chain_equalities` Chain tree; all deadlines are exactly tight. Tests that the DP uses $\leq$ rather than $<$, on a non-trivial path.

`secret06_nearest_greedy_trap` 3-node star; the nearest alarm has a loose deadline, but the farther one has a tight deadline that is missed if the near alarm is visited first. Catches "visit nearest first" greedy approaches.

`secret07_deadline_greedy_trap` 3-node chain; visiting the alarm with the smallest deadline first makes the other alarm impossible. The correct order is reversed. Catches EDF (earliest-deadline-first) greedy approaches.

`secret08_shared_ancestors` Two alarms in different subtrees; the path must go through their LCA. Tests correct use of tree distances.

`secret09_individually_reachable_but_impossible` Each alarm is reachable individually from S within its deadline, but no ordering visits both within their deadlines. Tests that the DP correctly identifies infeasibility despite individual feasibility.

`secret10_k18_star_dense_states` $N = 19$, $K = 18$, star tree. Every alarm is on a distinct leaf; deadlines are very loose (10^{15}), so all $2^{18} \times 18$ DP states are reachable. Deadline values up to 10^{15} test 64-bit arithmetic. Answer is the minimum over all 18! valid orderings.

`secret11_large_chain_stress` $N = 10\,000$, $K = 18$, chain tree. Long travel distances; tests that tree-distance computation scales to large N and forces 64-bit arithmetic.

`secret12_large_star_stress` $N = 200\,000$, $K = 18$, star tree. Maximum N with all alarms on leaves of a single hub. Tests memory and time on the largest star-shaped input.

`secret13_large_binary_stress` $N = 200\,000$, $K = 18$, balanced binary tree. Structured but nontrivial distances; tests LCA / distance computation at scale.

`secret14_medium_random_feasible` $N = 5\,000$, $K = 18$, random tree, feasible answer. Random structure reduces the chance of accidentally correct greedy solutions.

`secret15_medium_random_mixed_tight` $N = 5\,000$, $K = 18$, random tree, impossible answer. Mixed tight and loose deadlines; tests that the DP correctly returns `IMPOSSIBLE` on a realistic random instance.

Why These Tests Are Sufficient

The test suite covers:

- **Trivial and edge cases:** $K = 0$, $K = 1$, start room alarmed, exact deadline equality.
- **Correctness under greediness:** two dedicated greedy-trap cases catch the two most natural wrong approaches.
- **Feasibility vs. infeasibility:** multiple cases of each, including one where individual reachability does not imply joint feasibility.
- **Tree shapes:** chain, star, balanced binary, random – each stresses distance computation differently.
- **Scale:** N up to 200 000 and $K = 18$ in separate cases, together confirming that both the DP and the tree preprocessing run within the time limit.
- **64-bit arithmetic:** large edge weights ($\leq 10^9$) and large deadlines ($\leq 10^{15}$) appear throughout.

Test Case Generation Strategy

Test cases were produced by a combination of manual construction and the Python generator in `test_case_generator/gen.py`.

1. **Manual construction** (samples, edge/greedy-trap cases): we hand-chose small trees and alarm parameters to make the desired property as transparent as possible.
2. **Structured generators** (chain, star, binary, random): `gen.py` produces trees of the desired shape, then picks K alarm rooms uniformly at random. A random visiting order is simulated to compute exact finish times, and deadlines are set to the finish time (tight) or finish time plus a random slack (loose).
3. **Answer verification:** all `.ans` files were produced by running both accepted solutions (LCA-based and BFS-based) and confirming they agree on every test case.